PROJECT AEGIS

Secure Personal AI Operating System

"An intelligent, secure, privacy-first operating companion that augments human productivity through natural interaction, autonomous task execution, and adaptive learning while maintaining complete user control."

Product Requirements & Software Design Specification

Project Codename: AEGIS

Assistant Name: JARVIS

Version: 1.0 (Release Candidate)

Author: Varad Palod

Date: July 2026

Core Technology Stack:
Python • FastAPI • LangGraph • PostgreSQL • ChromaDB • Whisper • OpenCV • React • Playwright

1. Executive Summary

1.1 Vision & Mission

Vision: To create an omnipresent, privacy-respecting AI operating companion that seamlessly bridges the gap between human intent and digital execution.

Mission: To engineer a system that does not merely respond to prompts but actively manages, plans, and executes complex workflows across file systems, browsers, and secure databases without compromising user data integrity or privacy.

1.2 Core Philosophy

Unlike cloud-dependent commercial LLM platforms (ChatGPT, Claude) or rudimentary voice assistants (Siri, Alexa), Project AEGIS is designed as a sovereign intelligence node. The architecture prioritizes:

- **Local Security:** Crucial data processing happens locally, with strict permission layers before agent execution.
- **Autonomous Reasoning:** Utilization of advanced Planner models (via LangGraph) to break down complex queries into sequential tool calls.
- **Long-term Memory:** Vector databases (ChromaDB) to map user preferences, context, and historical interactions seamlessly.

2. Feature Specifications

The following features represent the core interactive capabilities of JARVIS.

Feature	Description	Priority	Security Level
Voice Commands	Natural speech interaction powered by Whisper. Continuous listening with wake-word detection.	High	Level 1
Face Authentication	OpenCV-based facial recognition to unlock Level 3/4 commands and sensitive data.	Medium	Level 3
File System Control	Read, write, organize, and search local files contextually based on user commands.	High	Level 2
Browser Automation	Playwright-driven autonomous web surfing, research, and data extraction.	High	Level 1
Coding Assistant	Sandbox execution, syntax validation, git integration, and autonomous debugging.	High	Level 2
Vision System	Screen analysis and real-world camera analysis to parse context for the AI reasoning engine.	Medium	Level 2

3. Functional Requirements (FR)

The system must adhere to the following deterministic functional requirements to ensure correct operational behavior.

> **FR-001: Multi-Factor AI Authentication**
>
> The system shall authenticate the user via Voice Print and Facial Recognition before allowing execution of any privileged commands (e.g., file deletion, vault access).

FR-002: Autonomous Task Planning

Upon receiving a complex prompt, the Planner Module shall decompose the request into a DAG (Directed Acyclic Graph) of sub-tasks and assign them to specialized Agents.

FR-003: Contextual Memory Retrieval

The system shall query the ChromaDB vector database prior to generating a response to append relevant historical user context to the LLM prompt window.

FR-004: Tool Sandbox Execution

All auto-generated code execution must occur within an isolated Docker container or constrained sandbox environment with no unmonitored host network access.

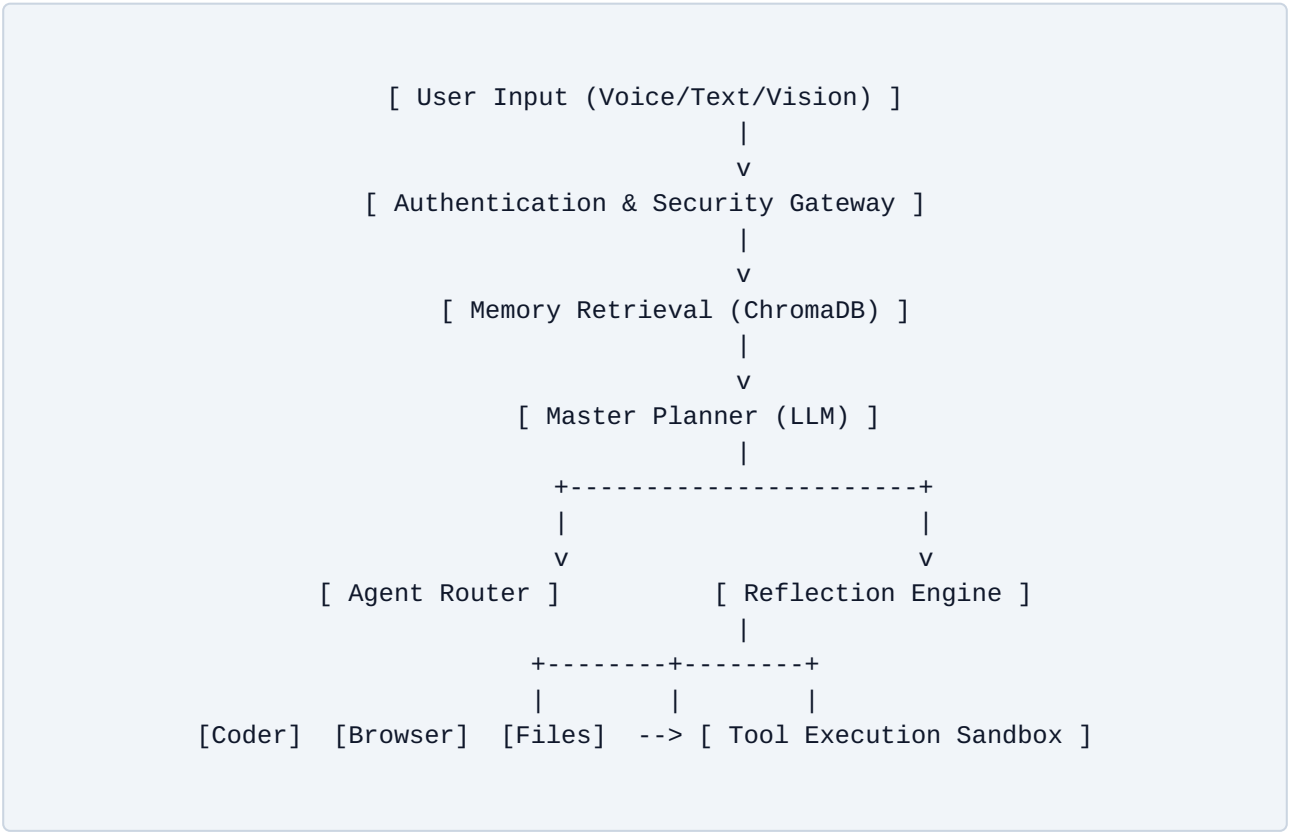
4. Non-Functional Requirements (NFR)

Category	Requirement ID	Description	Target Metric
Performance	NFR-001	Voice-to-Text latency for wake-word and short commands.	< 500ms
Security	NFR-002	Data encryption for the local SQLite/ PostgreSQL memory tables.	AES-256
Reliability	NFR-003	System uptime in background daemon mode.	99.9%
Offline Capability	NFR-004	Core conversational and file OS commands must function offline via local LLM.	Zero Cloud Dependency

5. System Architecture

5.1 Overview

The architecture of Project AEGIS follows a strict pipeline designed to separate intent recognition, security validation, reasoning, and execution.



5.2 Core Modules

- **Security Gateway:** Intercepts all commands to evaluate threat level, prompt injection risks, and required permission tiers.
- **Planner:** The brain of the application. Uses LangGraph to establish multi-step reasoning chains.
- **Agent Router:** Routes the sub-tasks to specialized sub-agents (e.g., delegating web scraping to the Browser Agent).

6. Security Architecture

Security is the bedrock of Project AEGIS. Since the AI has system-level access, strict RBAC (Role-Based Access Control) is implemented.

6.1 Permission Levels

Level	Name	Access Rights	Authentication Required
Level 0	Public	General conversation, weather, web search.	None
Level 1	Standard	Read non-sensitive files, manage calendar, draft emails.	Voice Match
Level 2	Advanced	Write to file system, execute standard scripts.	Voice + PIN / JWT Session
Level 3	Admin	System configuration, application uninstalls.	Face Auth
Level 4	Vault	Access to encrypted secrets, API keys, financials.	Face Auth + Master Password

> **Threat Mitigation: Prompt Injection**
>
> All inputs are passed through a lightweight LLM classifier configured specifically to detect adversarial prompts, overrides, and jailbreak attempts before they reach the main Planner module. Malicious requests are instantly neutralized and logged in the immutable audit trail.

7. AI & Agent Architecture

7.1 Memory Engine

The memory architecture combines short-term buffer memory (for the active session) and long-term semantic memory (ChromaDB). Information extraction agents periodically summarize active sessions and embed key facts (e.g., "User prefers Python over Java") into the vector store.

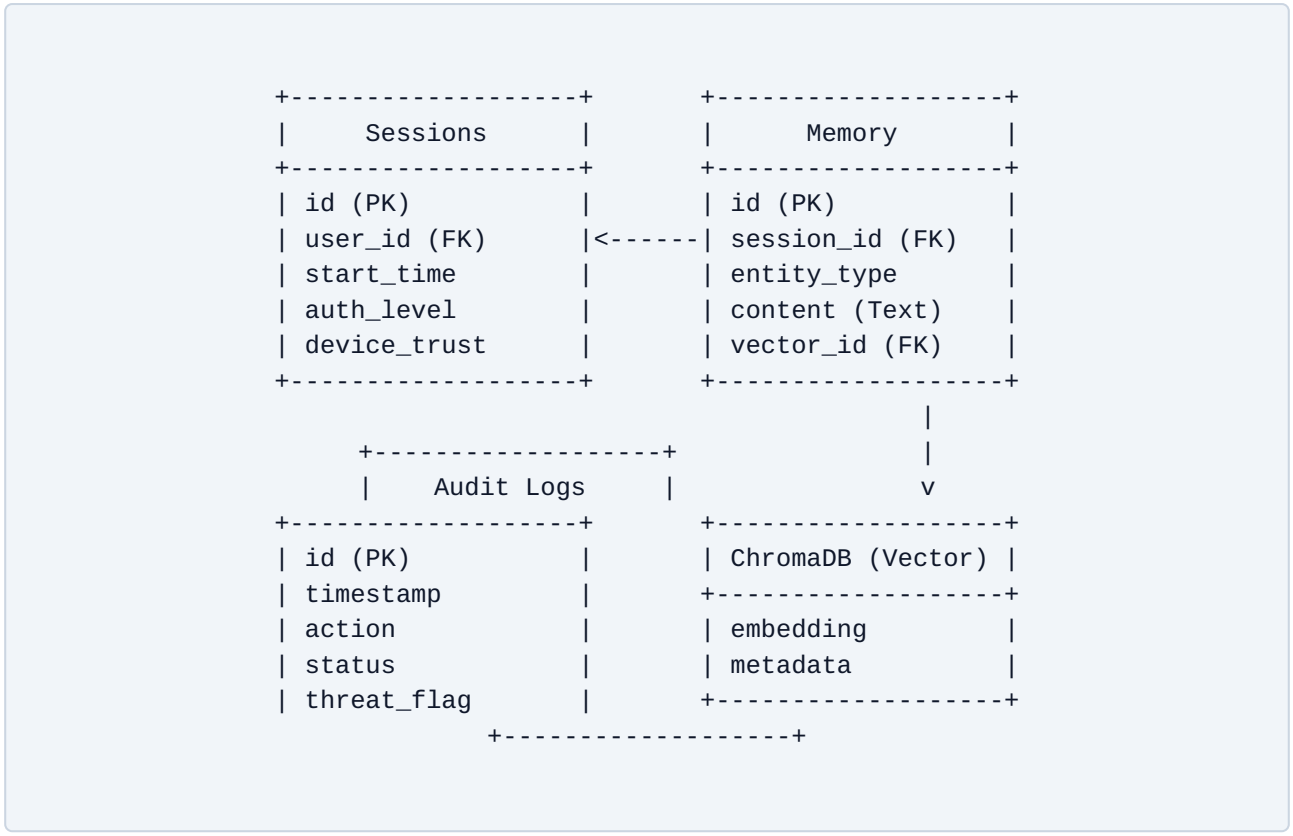
7.2 Agent Specialization

Agents are strictly scoped LLM instances equipped with specific tools:

- **Coding Agent:** Equipped with `subprocess`, `ast`, and `git`. Responsibilities include writing, linting, and testing code.
- **Browser Agent:** Uses Playwright to navigate DOM trees, click elements, and extract text/JSON from web pages.
- **File Agent:** Uses Python `os` and `shutil` libraries. Restricted to user-defined safe directories unless elevated.

8. Database Design

The system utilizes PostgreSQL for structured transactional data and ChromaDB for vector embeddings.



9. Development Roadmap

The project will be developed in iterative versions to ensure stability of core features before expanding capability.

Phase	Version	Deliverables
Foundation	v0.1 - v0.2	PostgreSQL Setup, Basic Auth, Local LLM Integration, Voice Pipeline (Whisper).
Cognition	v0.3 - v0.4	ChromaDB Memory implementation, LangGraph Planner, Basic Tool Calling.
Execution	v0.5 - v1.0	File System integration, Playwright Browser Agent, Release Candidate UI.

Expansion	v2.0 - v3.0	Vision System (OpenCV), Multi-Agent Orchestration, Wearable integration.

10. Testing Strategy

To guarantee reliability and security, testing will encompass:

- **AI Testing:** Benchmarking prompt adherence and evaluating hallucination rates using standard test datasets.
- **Security Testing:** Penetration testing the Tool Runtime sandbox to ensure code execution cannot escape the Docker container.
- **Voice/Face Accuracy:** Testing false-positive and false-negative rates of biometric authentication under varying ambient noise and lighting conditions.